DIPLOMA IN INFORMATION TECHNOLOGY

PROGRAMMING FUNDAMENTAL (DIT1124)

GROUP ASSIGNMENT

ASSIGNMENT TITLE:

Smart Workshop Energy & Safety Monitoring System

We hereby certify that this assignment is our work and where materials have been used from other resources, they have been properly acknowledged. We also understand that we will face the possibility of failing the module if the content of this assignment is plagiarized.

No.	Name	Student ID	Signature
1	ERIC DAREN ERLANDA	SCSJ2400425	
2	MOHAMMAD AMIN DARRIS	SCSJ2500448	
3	TAN JIN YUAN	SCSJ2500372	
4			

Date:_____

LECTURER NAME : **NUR DIANA MADINAH BINTI AB HADI**
RELEASE DATE : **26th JANUARY 2026**
SUBMISSION DATE : **08th MARCH 2026**
PRESENTATION DATE

MARK OBTAINED:

Contents

1. Scenario 3

PART A: Case Study Analysis 4

Objective of the Smart Workshop Energy and Safety Monitoring System (SWESMS) .. 4

PART B: IPO Analysis, Algorithm & Pseudocode 7

Overall System IPO 7

Combined Overall IPO 9

ALGORITHM 11

PART C: Modular Design & Flowchart 12

FULL SYSTEM PSEUDO CODE 12

FLOWCHART 18

PART D: Application Features and Testing 20

Part E: Consistency and Enhancement 24

Part F: Reflection 25

ASSIGNMENT BRIEF

1. Scenario

The Engineering and IT workshop is a shared space that supports practical classes and student projects. It contains equipment such as soldering stations, 3D printers, lab PCs, networking devices, and other electrical tools. Currently, the workshop records usage are used manually recorded, making it difficult to estimate daily electricity cost, identify high-energy equipment, and detect safety risks early.

You are assigned as a development team to design and build a console-based Smart Workshop Energy & Safety Monitoring System (SWESMS) using C++ in Eclipse. The system will record equipment information, accept daily updates (hours used and temperature readings), calculate energy consumption (kWh) and daily cost (RM), and generate safety and maintenance alerts such as overheat, near-overheat, and maintenance due. No hardware integration is required; all readings are simulated through user input.

As developers, you must begin by analyzing the problem using IPO (Input–Process–Output) analysis, then design the solution using algorithm, pseudocode and flowcharts before coding. Your final submission must include the working C++ program, output evidence, saved report file, GitHub repository link, peer-to-peer evaluation, and a presentation with a live demo.

INTRODUCTION:

The Engineering and IT Workshop plays a crucial role in supporting hands-on learning, practical classes, and student projects by providing access to essential equipment such as soldering stations, 3D printers, laboratory computers, networking devices, and various electrical tools. However, the current practice of manually recording equipment usage makes it difficult to accurately monitor daily electricity consumption, estimate operating costs, identify high-energy-consuming equipment, and detect potential safety risks such as overheating at an early stage. This limitation can lead to inefficient energy management, increased costs, and higher safety risks within the workshop environment.

To address these challenges, the Smart Workshop Energy & Safety Monitoring System (SWESMS) is proposed. This system is a console-based application developed using C++ in Eclipse that simulates smart monitoring through user-input data. SWESMS is designed to record equipment information, accept daily usage hours and temperature readings, calculate energy consumption in kilowatt-hours (kWh) and daily electricity cost in Ringgit Malaysia (RM), and generate safety and maintenance alerts such as overheat warnings, near-overheat notifications, and maintenance reminders. By implementing this system, the workshop can improve energy efficiency, enhance safety awareness, and support better decision-making through structured data analysis and reporting

PART A: Case Study Analysis

The Engineering and IT workshop currently records equipment usage manually. This manual approach creates several challenges while monitoring:

- Records are inconsistent, time-consuming, and prone to human error.
- Difficulty in estimating **daily electricity consumption and cost.**
- No clear identification of high-energy-consuming equipment.

Safety risks such as **overheating equipment** are not detected early.

Objective of the Smart Workshop Energy and Safety Monitoring System (SWESMS)

SMART 1:

Specific	The System will measure the daily energy consumption (kWh) and the electricity cost (RM) for each workshop equipment based on power rating and the usage hour.
Measurable	Energy usage is measure in kilowatt hour (kWh) and cost is measured in Ringgit Malaysia (RM).
Achievable	The calculation can be performed by user inputting the data in the console program without hardware integration.
Relevant	This will help the workshop estimate their daily electricity expenses and identify the high energy equipment.
Timely/Time-Bound	The calculation is performed every team daily usage data is enter,

Table 1, 1st SMART Objcctive

SMART 2:

Specific	The system will monitor the equipment temperature and generate near overheat and overheat alerts.
Measurable	Near overheat is detected at temperature ≥70°C and overheat at temperatures ≥80°C.
Achievable	Temperature values are entered manually and evaluated using conditional statements in C++.
Relevant	Early detection of overheating reduces safety risks and potential equipment damage.
Timely/Time-Bound	Safety alerts are generated immediately after temperature input for each day.

Table 2, 2nd SMART Objective

PACT Target User

P, People	The primary users of SWESMS are workshop technicians and lab assistant who are responsible to monitor the engineering and IT workshops
A, Activity	The Users will perform data entry task such as inputing equipment IDs, current equipment temperatures and the usage durations
C, Context	The context of usage is mostly relied on the enviroment, specifically in engineering and IT workshops where high energy tools resides.
T, Technology	The technology chosen for SWESMS is a menu-driven console-based application developed using C++ in Eclipse IDE that supports structured programming concepts such as loops, selection and modular functions for energy calculation and safety monitoring

Table 3, PACT Target User

Summary:

The primary users of SWESMS are workshop technicians and lab assistants who monitor engineering and IT workshops. They perform data entry tasks such as inputting equipment IDs, temperatures, and usage durations to track equipment performance. The system is used in engineering and IT workshop environments where high-energy equipment operates. SWESMS is developed as a menu-driven console-based application using C++ in Eclipse IDE to support energy consumption calculation and safety monitoring.

PART B: IPO Analysis, Algorithm & Pseudocode

1. Construct an IPO Analysis for SWESMS (overall system) and at least **TWO (2)**

Overall System IPO

Component	Description
Input	• Tariff rate • Number of equipment • Equipment ID, Name, Power Rating (Watts) • Daily Usage Hours • Temperature Reading (°C) • Total Usage Days
Process	• Convert power from Watts to kW. • Calculate energy consumption (kWh). • Calculate daily electricity cost (RM). • Check safety status (Normal, Near-Overheat, Overheat). • Check maintenance status. • Validate inputs (non-negative, reasonable range). • Save results to file • Display summary
Output	• Display daily usage summary per equipment • Energy consumption (kWh) • Daily cost (RM) • Safety alerts • Maintenance alerts • Report file(workshop_report.txt)

Table 4, Overall IPO

The first IPO shows the overall on how the SWESMS work, it make sures that all of the necessary information were recorded/stored such as equipment details, usage hours, temperature reading and the electricity cost. The output shows the expected outcome of the SWESMS like maintenance alerts, daily ussage, energy consumption and safety alerts

Module 1: Energy Consumption & Cost Calculation

Component	Description
Input	• Power (Watts) • Daily Usage Hours • Electricity Tariff (RM/kWh)
Process	• Convert Power to kW: Power_kW = Power_Watts / 1000 • Calculate Energy: Energy_kWh = Power_kW * Usage_Hours • Calculate Cost: Cost_RM = Energy_kWh * Tariff • Validate that hours ≥ 0, Power > 0, Tariff > 0
Output	• Energy consumed(kWh) • Daily electricity cost (RM)

Table 5, 1st Module IPO

The first module focuses on monitoring electricity usage by requiring power ratings, usage hours and electricity tariff rates as inputs. These input are compulsory to calculate energy consumption in kWh and the price of each usage. Each processes ensure the correct calculations, meanwhile output helps to identify high energy consuming equipment, support cost control and energy efficiency in the workshop

Module 2: Safety & Maintenance Monitoring

Component	Description
Input	• Temperature (°C) • Usage Days
Process	• Check temperature: 1. >80°C → Overheat 2. 70–80°C → Warning 3. <70°C → Normal • Check maintenance: UsageDays ≥ 30 → Maintenance Due, else OK • Validate temperature within 0–100°C. • Validate usage days ≥ 0.

Output	• Safety status (Normal / Near-Overheat / Overheat) • Maintenance alert (OK / Due)

Table 6, 2nd Module IPO

The second module requires usage hours and temperature readings to track equipment condition overtime. The inputs allow the SWESMS to determine whether equipment has exceeded safe operating limits or rqeuires maintenance ASAP. The processes compare usage hours/days and temperature against preset limits to generate timed-alerts while the output helps prevent equipment failure, reduce safety risk and support maintenance planning.

Combined Overall IPO

Component	Description
Input	• Equipment id • Equipment name • Power rating (Watts/W) • Daily usgae hours • Total usage hours • Temperature reading (Clecius/C) • Electricity tariff rate • Maintenance usage hours
Process	• Input validation (from input) -Power rating > 0 -Daily usage hours >= 0 -Electricity tariff rate > 0 -Maintenance limit > 0 -Temperature >= 0 • Store and update equipment information • Convert power rating from Watt/W to kiloWatt/kW • Calculate daily electricity cost • Update total usage hours • Compare temperature with safety limit

	- >= 60 C = Warning - >= 80 C = Overheat • Compare accumulated usage with maintenance limit • Generate energy, safety and maintenance alerts • Save report to file (if doing bonus question) • Power (kW) = power (W) / 1000 • Energy consumption = power (W) / 1000 * usage hours • Electricity cost = energy consumption * tariff rate • Total usage hours = previous usage + daily usage
Output	• Daily energy consumption per equipment • Daily electricity cost per equipment • Safety status (Normal / Warning / Critical) • Maintenance status • Display results (in Eclipse console) • Saved energy and safety report file (bonus question)

Table 7, Combined IPO

The final IPO combines energy consumption, safety detection and maintenance management into a single workflow. By combining all of the required inputs and processes, the system ensures consistent data handling and accurate analysis across modules. The final outputs provide a comprehensive daily overview of energy usage, costs, safety status and maintenance needs, enabling efficient monitoring, safer operations and better resource management within the engineering and IT workshop.

ALGORITHM

Select date

1: Input tariff rate

2: Input user hour used

3: Input temperature reading

4: Calculate the hour rate of usage

kWh = (Watts / 1000) * hours used

Check if Peak Surcharge Rule by applying an additional 20% if total daily energy exceeds 8 kWh

5: Process the temperature reading (hot or not)

-if USAGE is Smaller or equal to 80 = NORMAL

-if USAGE is Smaller or equal to 90 = WARNING

-if USAGE is Bigger or equal to 95 = CRITICAL

6: Check if the equipment needs to be serviced (Hour rate usage example; 80 hour of usage)

-Checking by getting the data from the hour used for the hour rate

PART C: Modular Design & Flowchart

FULL SYSTEM PSEUDO CODE

```
START

DECLARE equipmentList as empty list
SET tariffRate = 0.50

REPEAT
   DISPLAY:
      1. Add Equipment
      2. View Equipment
      3. Update Equipment
      4. Delete Equipment
      5. Daily Update
      6. Service Equipment
      7. Export to CSV
      8. Exit

   INPUT choice

   IF choice = 1 THEN
      INPUT numberOfEquipment

      FOR i = 1 TO numberOfEquipment
         CREATE new Equipment

         REPEAT
            INPUT equipmentID
            CONVERT equipmentID to uppercase
         UNTIL equipmentID does not exist in equipmentList

         INPUT equipmentName
```

```
        INPUT powerRating
        INPUT maintenanceLimit

        SET totalUsage = 0
        SET records = empty

        ADD Equipment to equipmentList
    END FOR
END IF


IF choice = 2 THEN
    IF equipmentList is empty
        DISPLAY "No equipment available"
    ELSE
        FOR each equipment in equipmentList
            DISPLAY equipment details
            DISPLAY totalUsage

            IF totalUsage ≥ maintenanceLimit
                DISPLAY "MAINTENANCE DUE"
            ELSE
                DISPLAY "NOT DUE"
            END IF

            FOR each dailyRecord
                DISPLAY day, usageHours, temperature,
                        energyKWh, cost, safetyStatus
            END FOR
        END FOR
    END IF
END IF
```

```
IF choice = 3 THEN
   INPUT equipmentID
   FIND equipment

   IF found THEN
      INPUT newID (must not duplicate)
      CONVERT to uppercase
      UPDATE equipment ID

      INPUT new powerRating
      INPUT new maintenanceLimit
   ELSE
      DISPLAY "Equipment not found"
   END IF
END IF


IF choice = 4 THEN
   INPUT numberToDelete

   FOR i = 1 TO numberToDelete
      INPUT equipmentID
      FIND equipment

      IF found
         REMOVE equipment from list
      ELSE
         DISPLAY "Equipment not found"
      END IF
   END FOR
END IF


IF choice = 5 THEN
```

```
    INPUT equipmentID
    FIND equipment

    IF found THEN
        SET day = number of existing records + 1

        INPUT usageHours
        INPUT temperature

        CALCULATE energyKWh =
            (powerRating / 1000) × usageHours

        CALCULATE cost =
            energyKWh × tariffRate

        IF temperature ≥ 80
            safetyStatus = "OVERHEAT"
        ELSE IF temperature ≥ 60
            safetyStatus = "NEAR-OVERHEAT"
        ELSE
            safetyStatus = "NORMAL"
        END IF

        ADD usageHours to totalUsage
        STORE daily record
    ELSE
        DISPLAY "Equipment not found"
    END IF
END IF


IF choice = 6 THEN
    INPUT equipmentID
    FIND equipment
```

```
    IF found THEN
      IF totalUsage = 0
        DISPLAY "No service needed"
      ELSE IF totalUsage < maintenanceLimit
        DISPLAY "Maintenance not due"
      ELSE
        SET totalUsage = 0
        DISPLAY "Equipment serviced"
      END IF
    ELSE
      DISPLAY "Equipment not found"
    END IF
  END IF


  IF choice = 7 THEN
    OPEN file "SWESMS_Report.csv"

    WRITE header line

    FOR each equipment
      FOR each dailyRecord
        WRITE equipment and record data to file
      END FOR
    END FOR

    CLOSE file
    DISPLAY "Export successful"
  END IF

UNTIL choice = 8

DISPLAY "Exiting system"
```

END

2nd Module

```
MODULE MonitorSafetyAndMaintenance(
   temperature, usageHours, maintenanceLimit,
   safetyStatus, maintenanceStatus
)

IF temperature >= 80 THEN
   safetyStatus = "OVERHEAT"
ELSE IF temperature >= 60 THEN
   safetyStatus = "NEAR-OVERHEAT"
ELSE
   safetyStatus = "NORMAL"
END IF


IF usageHours >= maintenanceLimit THEN
   maintenanceStatus = "MAINTENANCE DUE"
ELSE
   maintenanceStatus = "NOT DUE"
END IF

END MODULE
```

FLOWCHART

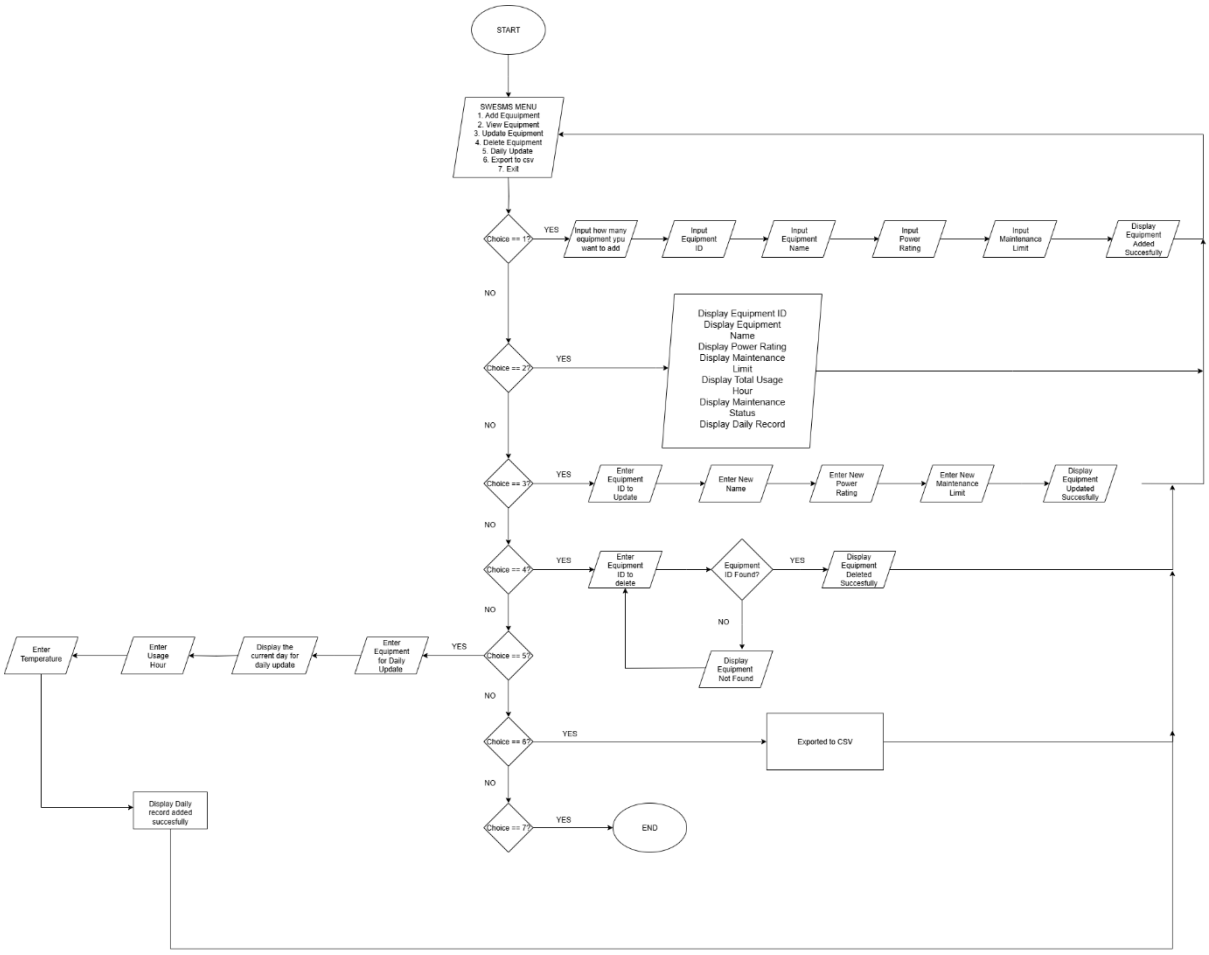

Figure 1, Flowchart of the overall system

Justification of the system flowchart (SWESMS equipment management system):

This flowchart shows how the SWESMS equipment management system works. It is made to organize and manage equipment data in a smart and successful way. The system has a menu-driven structure that lets users do different things by making clear choices. This makes it easy to use, accurate and gives them control.

Structured Menu System

This structure makes the system easy to use, navigate and organize, which cuts down on mistakes and confusion.

1. Add equipment function

This makes sure that all important information is saved before it is saved, keeping the data complete and consistent. A confirmation message ("Equipment Added Successfully") makes sure that users get feedback and that the system works properly.

2. View equipment function

This provides full transparency of equipment status, supporting monitoring, planning, and decision-making.

3. Update equipment function

This keeps the system credible and up to date, so you can make changes and corrections without deleting records that are already there.

4. Delete equipment function

This keeps data safe and prevent it from being lost by accident.

5. Daily update function

This helps with daily monitoring and tracking of performance, which is useful for preventive maintenance and operational analysis.

6. Export to CSV

This improves system integration and data portability.

PART D: Application Features and Testing

1.

```
========== SWESMS MENU ==========
1. Add Equipment
2. View Equipment
3. Update Equipment
4. Delete Equipment
5. Daily Update
6. Service Equipment
7. Export to CSV
8. Exit
Enter choice: 1
How many equipment to add?:  2

Entering Equipment 1
Enter Equipment ID: S01
Enter Equipment Name: Power drill
Enter Power Rating (W): 70
Enter Maintenance Limit (Hours): 3
Equipment added successfully!

Entering Equipment 2
Enter Equipment ID: S02
Enter Equipment Name: JackHammer
Enter Power Rating (W): 90
Enter Maintenance Limit (Hours): 7
Equipment added successfully!
```

2.

```
========== SWESMS MENU ==========
1. Add Equipment
2. View Equipment
3. Update Equipment
4. Delete Equipment
5. Daily Update
6. Service Equipment
7. Export to CSV
8. Exit
Enter choice: 2

====== Equipment 1 ======
ID: S01
Name: Power drill
Power: 70 W
Maintenance Limit: 3 hrs
Total Usage: 0 hrs
Maintenance Status: NOT DUE
No daily records available.

====== Equipment 2 ======
ID: S02
Name: JackHammer
Power: 90 W
Maintenance Limit: 7 hrs
Total Usage: 0 hrs
Maintenance Status: NOT DUE
No daily records available.
```

3.

```
========== SWESMS MENU ==========
1. Add Equipment
2. View Equipment
3. Update Equipment
4. Delete Equipment
5. Daily Update
6. Service Equipment
7. Export to CSV
8. Exit
Enter choice: 5
Enter Equipment ID for daily update: S01
Day 1 update for Power drill
Enter usage hours: 3
Enter temperature (°C): 90
Daily record added successfully!
```

4.

```
========== SWESMS MENU ==========
1. Add Equipment
2. View Equipment
3. Update Equipment
4. Delete Equipment
5. Daily Update
6. Service Equipment
7. Export to CSV
8. Exit
Enter choice: 2

====== Equipment 1 ======
ID: S01
Name: Power drill
Power: 70 W
Maintenance Limit: 3 hrs
Total Usage: 3 hrs
Maintenance Status: MAINTENANCE DUE

--- Daily Usage Records ---
Day 1 | Usage: 3 hrs | Temp: 90 °C | Energy: 0.21 kWh | Cost: RM 0.105 | Safety: OVERHEAT

====== Equipment 2 ======
ID: S02
Name: JackHammer
Power: 90 W
Maintenance Limit: 7 hrs
Total Usage: 0 hrs
Maintenance Status: NOT DUE
No daily records available.
```

5.

```
========== SWESMS MENU ==========
1. Add Equipment
2. View Equipment
3. Update Equipment
4. Delete Equipment
5. Daily Update
6. Service Equipment
7. Export to CSV
8. Exit
Enter choice: 4
How many equipment do you want to delete? 1

Enter Equipment ID to delete: S02
Equipment deleted successfully.
```

6.

```
========== SWESMS MENU ==========
1. Add Equipment
2. View Equipment
3. Update Equipment
4. Delete Equipment
5. Daily Update
6. Service Equipment
7. Export to CSV
8. Exit
Enter choice: 6
Enter Equipment ID for service: S01
Equipment Power drill has been successfully serviced.
```

7.

```
========== SWESMS MENU ==========
1. Add Equipment
2. View Equipment
3. Update Equipment
4. Delete Equipment
5. Daily Update
6. Service Equipment
7. Export to CSV
8. Exit
Enter choice: 7
Data exported to SWESMS_Report.csv successfully!
```

8.

```
========== SWESMS MENU ==========
1. Add Equipment
2. View Equipment
3. Update Equipment
4. Delete Equipment
5. Daily Update
6. Service Equipment
7. Export to CSV
8. Exit
Enter choice: 8
Exiting SWESMS...
```

Overall remark from test cases:

- Overall the system is good
- Preferablly use GUI
- instead of put equipment manually, put it like in a list before hand and the user will just be able to type the equipment id instead of putting all of the info 1 by 1.

Part E: Consistency and Enhancement

Comparison Between Flowchart and Final Code

The flowchart and the final code match up because the program uses the same menu structure, decision logic and step-by-step processes as the flowchart. Each choice is made using conditional statements and the codes input, processing and output operations.

Two improvemnet for implementation

Improvement 1: Input Validation

To make sure that users enter correct and valid data, like positive numbers for usage and power rating, non-empty names and valid menu choices, input validation should be put in place. This will help the system avoid crashes, errors and improve overall reliability and user experience.

Improvement 2: Improve Search Efficiency

Using a better data structure, like a map or indexed storage instead of a simple linear search, can make searches faster and more accurate. This is especially true when there are a lot of records stored, which improves the overall performance and scalability of the system.

Justification

Testing and user feedback showed that invalid or wrong inputs can cause system errors, crashes, or confusion for users. Also, slow searching methods become less useful and take longer when there are a lot of records stored, so expanding input validation and search efficiency will make the system more reliable, faster, more scalable, and more satisfying for users.

Part F: Reflection

IPO Analysis, Algorithm, Pseudocode and Flowchart are related to each other in a certain order during the system development process. IPO analysis is the basis because it makes it clear what data is needed, how it will be processed, and what the results will be. IPO helps us organize inputs like power rating, usage hours, and temperature, set up processes like energy calculation and safety checking and figure out the outputs like electricity cost and alerts for our Smart Workshop Energy and Safety Monitoring System (SWESMS). The algorithm then turned the IPO into a set of logical steps. Using loops, selection, and validation, pseudocode turned this logic flow in a way that made it easy to understand how the program was put together before writing any C++ code. The flowchart that was made is very similar to the final code because they both use the same menu-driven structure, validation loops, and conditional decision-making. For instance, the flowchart shows the process of checking input before doing calculations, and the final C++ code uses “while” loops to make sure the input is valid. Both versions of the temperature decision branches are also the same. When you only use one variable or data that is not well-structured, it can be hard to scale. If you need more than one equipment record, single variables will replace the old data. This makes it harder to grow and less productive. Using structured data like arrays or structs would make it easy of multiple equipment records. One bug found was that the early version of the algorithm had the wrong logic for checking temperatures, which led to the wrong safety classification. For instance, a temperature of 85°C was sometimes put in the wrong category. The problem was that the conditions were not set up in the right order. This was fixed by first checking if it was ">=80" and then ">=60." A test case at 85°C showed that it correctly showed “OVERHEAT.” Eclipse made coding easy by adding debugging tools and syntax highlighting, and Githud made working together easy by adding version control and commit tracking. I helped by making the IPO tables and adding loops for input validation. My changes made sure that calculations were stable and that there were fewer errors at runtime, which made the final system more reliable and of higher quality.